IDS Evasion and Detection using Snort

Abhang Nitinkumar Mandwale
MSc in Computing in Applied Cyber Security
Technological University Dublin
Dublin, Ireland
A00047036@mytudublin.ie

***Abstract*—This paper explores a critical topic of intrusion detection systems (IDS). These systems are critical for organizations of all sizes as they provide critical information on what type of traffic is entering the internal network. As the name suggests, these systems help in detecting malicious traffic, which can then be flagged by SIEM systems. Here, we will be looking at a widely known IDS, Snort. This paper presents the deployment of Snort, configuration, evasion techniques, and ways to detect these evasion techniques.**

***Index Terms*—Cyber Security, Network Security, Intrusion Detection Systems, Snort**

I. Introduction

With the age of the Internet, there has been a rapid growth of cyber attacks over the years. It becomes difficult to know where the attacks come from and what type of attack is being performed. Threat actors are constantly finding ways to infiltrate as many systems as possible either by manually attacking systems or, more commonly, by automated methods. To understand what type of traffic is coming to our machines, we have a system that detects all types of traffic. This is exactly what intrusion detection systems do.

Intrusion Detection Systems (IDS) are systems built to monitor traffic to avoid any kind of malicious traffic, thus detecting cyber attacks and potentially violating security policies. These are deployed in tandem with firewalls and are connected to SIEM systems for log correlation. These systems are built only for detecting malicious traffic, whereas for prevention, intrusion prevention systems (IPS) are deployed. [1]

Following are the two main types of IDS systems depending on where they are deployed:

- Network-Based IDS (NIDS): These systems are deployed to capture traffic coming from the entire network.
- Host-Based IDS (HIDS): These systems are deployed to capture any changes made within the host machine.

Snort is a widely known open-source intrusion detection and prevention system. It can be classified into a network-based IDS. It works based on rules which are used to detect attacks like Denial-of-Service. With the use of community written rules, it is able to detect attacks like Eternal Blue and attacks on Log4j, showing the robustness and potent of Snort. Another strength of Snort is that it runs fully in terminal. This reduces the CPU overload that GUI-based systems bring, and also is key when we are accessing machines through SSH. [2]

II. Deployment

A. Victim Machine

For our victim machine, the one which will host Snort and the one which we will attack, a cloud virtual machine will be used. Here, we will be using DigitalOcean's Droplet to serve the purpose. This plays along with the Snort's strength of being fully terminal based. This machine is configured with Ubuntu as its host operating system.

B. Attack Machine

For our attack machine, we will utilize a virtual machine set up using Oracle's VirtualBox [3] on our local machine. This machine is configured with Kali Linux [4] as its operating system. It is a go-to option since we will be using this machine to exploit our victim machine and Kali Linux comes with security tools pre-loaded.

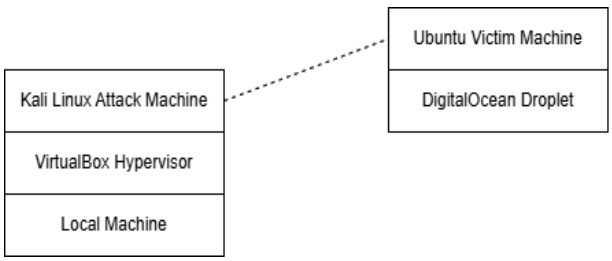


Fig. 1. Deployment Setup

III. Configuration

A. Victim Machine

Setting up Snort on our victim machine is fairly straightforward. The first thing we need to do is update all the existing packages that come with our OS using 'sudo apt-get update' and later we can install Snort simply using the command 'sudo apt-get install snort -y'. Once done we can verify correct installation using the command 'snort –version'. [5]

B. Attack Machine

To set up the attack machine, we would be using Oracle's VirtualBox. Any other hypervisor can also be used. We will then download the ISO image file for Kali Linux that we will load into our VirtualBox. It is recommended to download an older version for both VirtualBox and Kali to obtain as stable a setup as possible, and newest versions are usually prone to

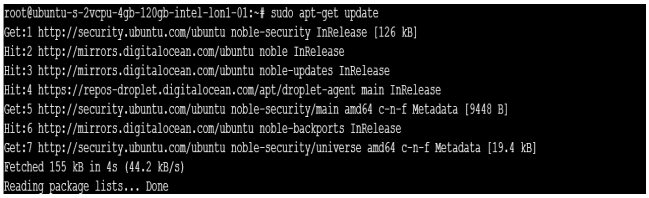


Fig. 2. Updating System Packages - Victim Machine

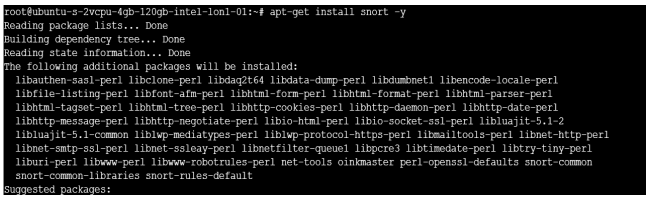


Fig. 3. Installing Snort - Victim Machine

bugs. This setup can be configured according to the user's need; however, we will give it enough RAM and storage for it to function without any hiccups.

C. Communication Establishment

For our victim machine, we need to setup firewall rules so that our Kali machine can communicate with our Ubuntu machine. Here, we will add the public IP address of our local network (obfuscated in the image), which we can check using 'icanhazip'. We will add rules for all the ports through which we will be attacking the victim machine.



Fig. 4. Firewall Configuration in DigitalOcean

For our attack machine, we need to make some changes in the VirtualBox network settings. We need to change the attachment type from NAT to Bridged Adapter. This is essential as this would make our Kali machine appear separate on our network, thereby being assigned its own IP. This will make the Kali machine act as a separate entity from our local machine.

To test if Snort is running, we will run it in self-test mode. This can be accessed by using the command 'snort -T -i eth0 -c /etc/snort/snort.conf', where snort will try and read all available rules. [11]

By default, Snort uses the snort.conf file as the main source for all rules, which are therefore used to detect traffic. Before that, we need to set the HOME_NET IP range as our local IP range.

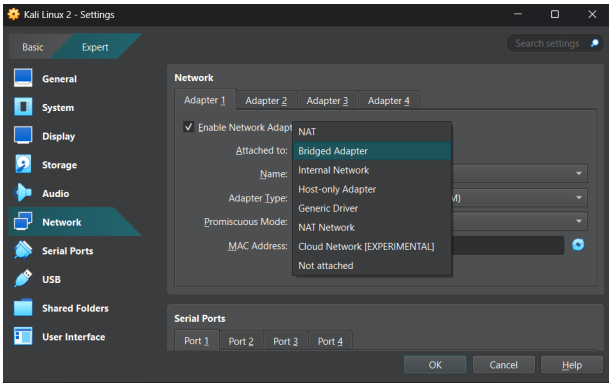


Fig. 5. Network Configuration in VirtualBox

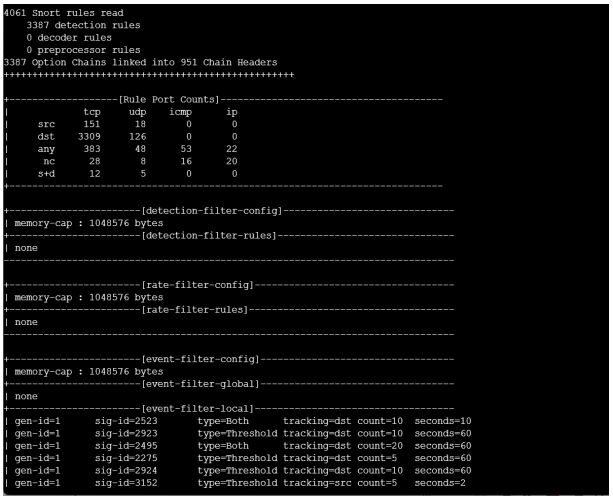


Fig. 6. Snort Self-Test Mode Output

IV. Evasion and Detection Techniques

A. Nmap Scan Detection

Reconnaissance is the first step of any cyber attack. Threat actors spend most of their time finding key information, openings, vulnerabilities that they could target. Based on that they decide how to move ahead. Nmap is a built-in tool within Kali which is widely used for port scanning. It is a powerful tool that can scan all 65,536 ports of any given IP

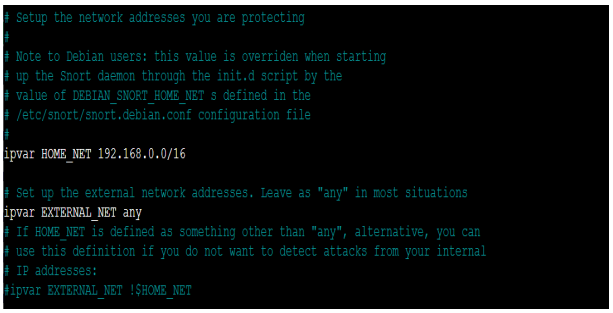


Fig. 7. Network Configuration in snort.conf

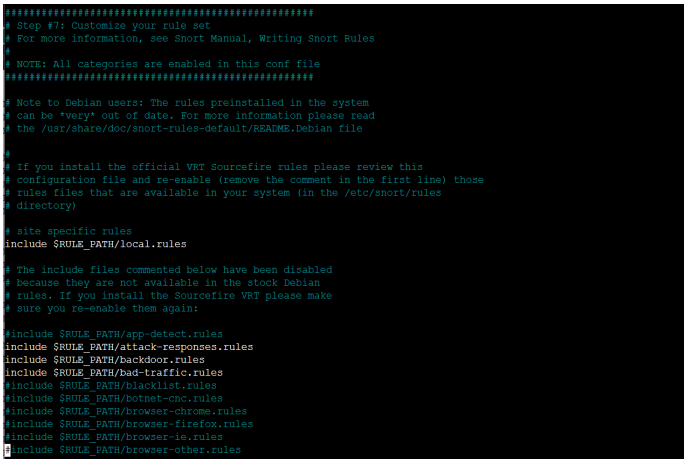


Fig. 8. Rule Set in snort.conf

address. Additionally, it would help us test the communication establishment that we did earlier. [6] [7]

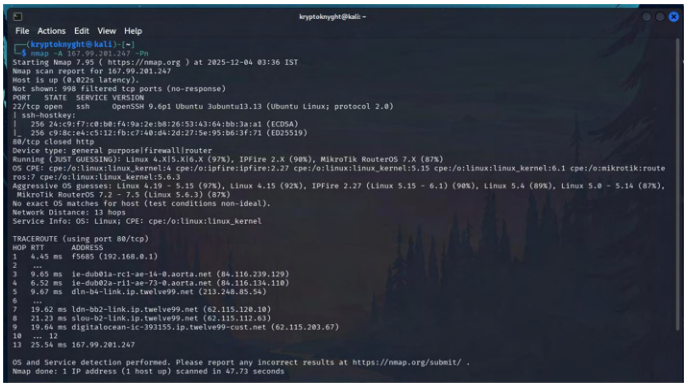


Fig. 9. Nmap Scan Initiation

After running the scan, we can see that port 22 is open. That means we can try to ssh into the victim machine from our attack machine. We will try running a brute-force command to check if we can find the password of our account.

We will set the rule in the local.rules file to detect Nmap scan, as shown in Fig.10. These scans fall under the ICMP category, so we will write the message accordingly. Additionally, if we ping to our victim machine, we will still receive the alert since ping also follows the ICMP protocol.

B. SSH Brute-Force Attempt

We previously found that port 22 on our victim machine is open. This gives us the opportunity to perform an SSH brute-force attack to find the credentials of our victim machine. [8]

However, since we have hosted the victim machine in the cloud, we are obligated to set a secure password. This could mean that brute-forcing may not actually give us the password. But we can still capture the brute-force attempt in Snort.

We will add the rule to detect any SSH connection attempt through port 22. This rule will be added to the local.rules file as shown in Fig.13.



Fig. 10. Rule for ICMP Detection



Fig. 11. Nmap Scan Detections in Snort

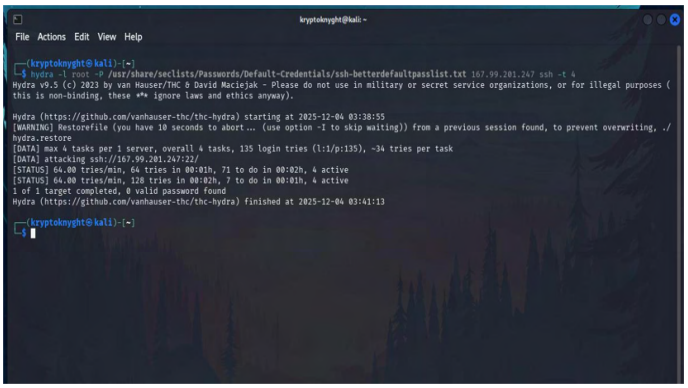


Fig. 12. SSH Credential Brute-Force Initiation

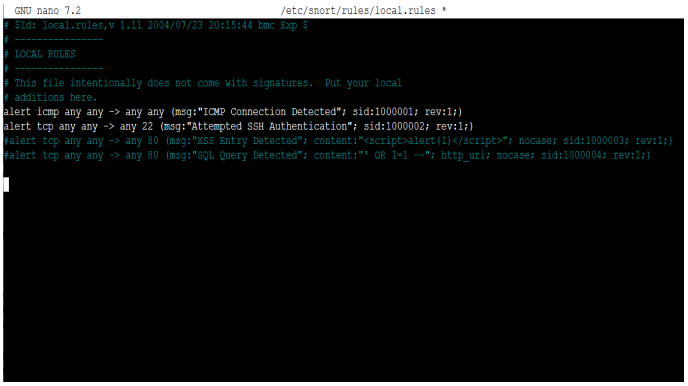


Fig. 13. Rule for SSH Traffic Detection

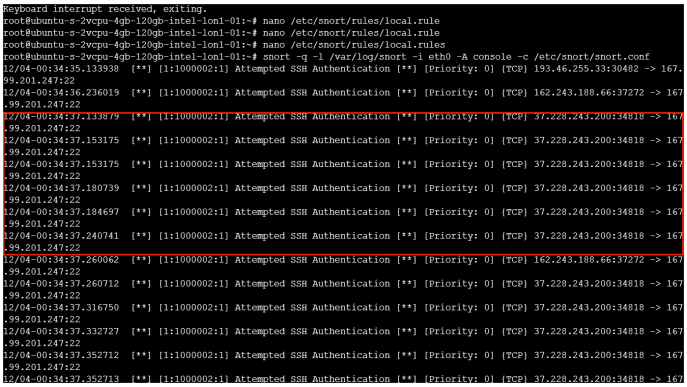


Fig. 14. SSH Traffic Captured in Snort

In Fig.14, we can see the incoming SSH traffic. The red highlighted box represents traffic from our attack machine when we tried to run the brute-force command against it. In addition, we can see the traffic coming from other source IPs. This traffic links to legitimate threat actors who are also trying to get into our victim machine. This happens because we have set up our machine on a cloud server with port 22 open to the whole internet. This can be mitigated by allowing only our local IP through DigitalOcean's firewall rules. Therefore, it is beneficial to have a secure password that cannot be cracked by brute-force systems.

C. Evasion using XSS Payload and Detection

We will look at one of the common attacks that is a Cross Site Scripting (XSS) attack. Cross Site Scripting is a type of injection attack for a web vulnerability where a threat actor is able to compromise a user's interaction with the web application. This takes place by sending malicious code to the browser in the form of a Javascript command. This attack can be used to extract critical information such as session cookies, confidential end user data, or modifying the presentation of the website. [9]

To implement this attack, we will first need to start a web server on our Ubuntu machine. This can be done using the command 'python3 -m http.server 80', where a web server is created on port 80. Now, this port can be seen open using the nmap command.

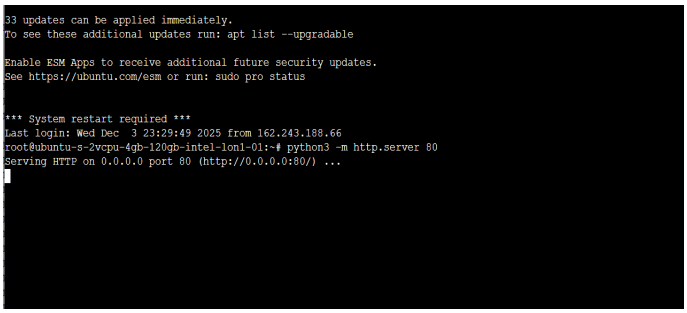


Fig. 15. Web Server Command on Ubuntu Machine

Here, We will be implementing the payload '<script>alert(1)</script>' which is commonly used to test web applications for the XSS vulnerability. We can set the rules in the local.rules file as shown in Fig.17. We will be detecting the payload based on the command <script>.

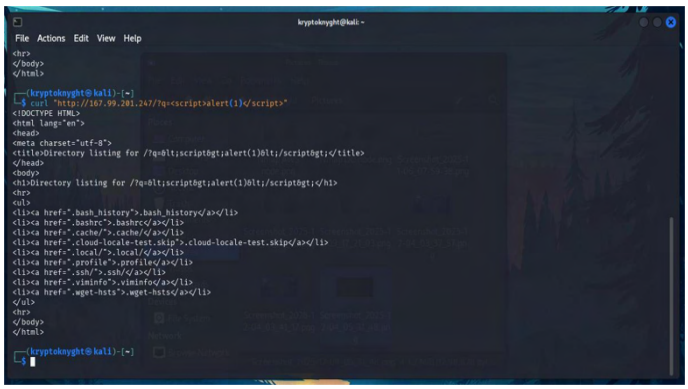


Fig. 16. Kali Terminal - XSS Payload

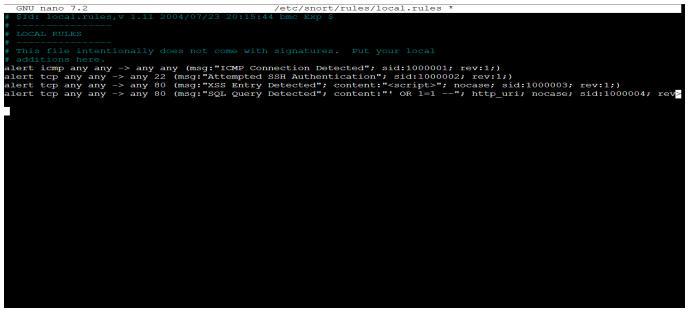


Fig. 17. Snort Rule for Detection on Port 80

We can see that even though we gave a part of the payload to check the traffic, Snort will be able to capture all the traffic with <script> as payload. This will cover many payloads that make use of the script tag.

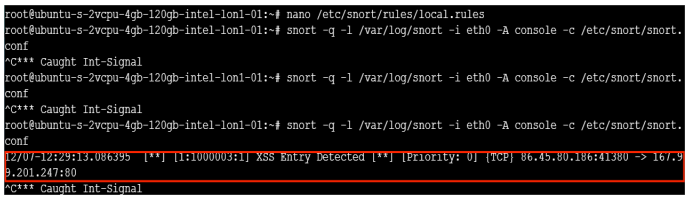


Fig. 18. XSS Payload Detected in Snort

D. Evasion using SQL Injection and Detection

Another attack that we can perform is SQL Injection. SQL Injection is a type of server side vulnerability which tricks databases into unauthorized access to accounts. This attack is commonly used on login entries, where the threat actor might be able to log into a user's account without knowing their

credentials. This also includes gaining admin access to web applications. [10]

For this attack, we will also need the server on port 80 to be running. For the payload, we will be using the `' OR 1=1 --` payload, which is commonly used by testers and threat actors to test SQL injection vulnerabilities. However, this payload is easy to catch, so threat actors use the obfuscated payload to bypass weak security measures. Therefore, we will use the payload `%27%20OR%201%3D1%20--` instead.

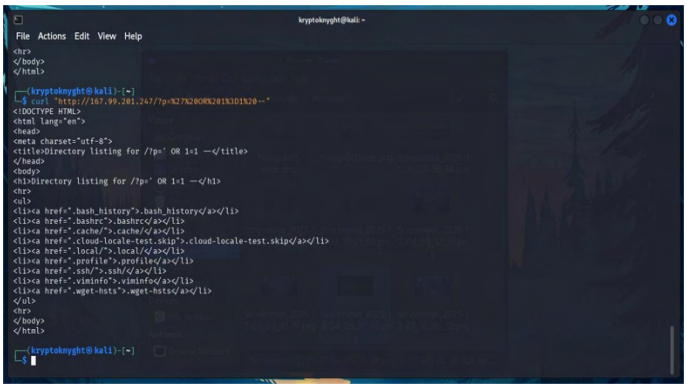


Fig. 19. Kali Terminal - SQL Injection

Given in Fig.17, we can see that the rule that is set for detection for SQL injection consists of the payload not being obfuscated. This will catch both the obfuscated payload and the non-obfuscated one.

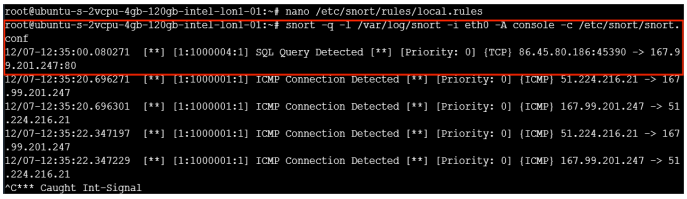


Fig. 20. SQL Injection Detected in Snort

V. Challenges and Learnings

A. Challenges

The main challenges faced during this activity were related to configurations and the establishment of communication between the two machines. Reconfiguring the Kali virtual machine from NAT to Bridged Adapter took some time to figure out and finding the public IP of self.

Another point here, not a challenge specifically, could be the open port 22 for all the internet. This was obvious as port 22 is most frequently targeted by threat actors. Due to the number of malicious SSH traffic that was being captured, finding the traffic from our Kali machine became a task. However, this can be fixed by allowing only the Kali machine's IP in the DigitalOcean firewall settings.

B. Learnings

Our setup here gave us a great idea of the configuration of Snort IDS. It tested the ability to make two machines communicate with each other irrespective of their geographical separation.

We also learn about the capabilities of intrusion detection systems, especially Snort, and how versatile and robust these open source tools can be for corporate networks.

C. Improvements

One of the main improvement points would be to explore deeper attack methodologies, which would include the use of Metasploit. An example here could be targeting the Eternal-Blue vulnerability in Windows systems. We could also review how the community written rules are engineered. These would teach us how to write better and more efficient rules.

Since Snort is also an intrusion prevention system, we could check out how malicious traffic can be proactively prevented instead of just detecting them. In addition, alerts can be fed into a SIEM or a SOAR system for faster and efficient detection of malicious traffic. With SOAR, we can add automation, such as automatically blacklisting malicious IPs and even raising incidences when required.

Conclusion

This activity is a great way to learn how computers talk to each other. In addition, we learn how we can detect any traffic that reaches our machines. This also raises concerns about the continuous monitoring and configuration management of the IDS systems that organizations should focus on. Revision of rules is critical for detecting and preventing new and upcoming attacks.

We can conclude that intrusion detection systems such as Snort are easy to work with and have great potential to detect much more complex attacks. Snort being open source also proves that implementing these devices can also be cost-efficient.

References

[1] Splunk, "Intrusion Detection Systems (IDS): Definition, Types, Purpose", Sep 3, 2024. https://www.splunk.com/en_us/blog/learn/ids-intrusion-detection-systems.html#:~:text=IDS%20types,and%20changes%20in%20regular%20traffic.

[2] Fortinet, "What is Snort?". https://www.fortinet.com/resources/cyberglossary/snort

[3] Oracle VirtualBox, "VirtualBox". https://www.virtualbox.org/

[4] OffSec, "Kali". https://www.kali.org/

[5] SkillsBuild Security, "Installing & Configuring SNORT Network Intrusion Detection System", Sep 3, 2025. https://www.youtube.com/watch?v=RSeAlWnZUWA&t=1205s&pp=ygUeaW50cnVzaW

[6] HackerSploit, "Intrusion Detection With Snort", May 20, 2022. https://youtu.be/Gh0sweT-G30?si=E27O5xIYZQCf4pfW

[7] Nmap, "Nmap.org". https://nmap.org/

[8] TrendMicro, "What is SSH Brute Force Attack And How To Deal With It", May 09, 2023. https://helpcenter.trendmicro.com/en-us/article/tmka-19689

[9] OWASP, "Cross Site Scripting (XSS)". https://owasp.org/www-community/attacks/xss/

[10] OWASP, "SQL Injection". https://owasp.org/www-community/attacks/SQL_Injection

[11] Ritwik Gupta, ”IDS Evasion Techniques - SNORT”. https://www.linkedin.com/in/ritwik-gupta-20198/details/projects/1706486525853/single-media-viewer/?profileId=ACoAAC-hU0kBdTUdBCjt-BY597vDnuBsnHK9Up0